

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

PART 22

Codeforces Roadmap

*The platform, the rating system, how to practice deliberately,
and a rating-by-rating skill map from unrated through 2000+*

PART 22

Codeforces Roadmap

Where everything from Parts 1-21 gets tested under a clock

** Where this Part fits**

This corresponds to the roadmap's Part XXIII / Phase 22 (Codeforces). It assumes the full technical toolkit from Parts 1-19 and the recognition process from Part 21 — this Part is about the platform, the practice discipline, and what's realistically expected at each rating level, not new algorithms.

Codeforces is the most widely used competitive programming judge for serious practice, largely because of its rating system: every rated contest updates a number that estimates your problem-solving skill relative to everyone else who competed. This Part explains how that system works, how to practice in a way that actually moves the number, and what's realistically expected of you at each rating band — without promising that finishing this book guarantees any particular rating. Rating depends on practice, contest reps, speed, and accuracy, same as any skill.

Contents of Part 22

- 1. The Platform: Rounds, Divisions, and Problem Difficulty
- 2. How the Rating System Works
- 3. How to Practice: Virtual Contests, Upsolving, Editorials
- 4. Contest Strategy: Reading, Ordering, Time Allocation
- 5. The Rating-by-Rating Skill Map (800 → 2000+)
- 6. Upsolving Training: Turning a Failure Into a Skill
- Part 22 Revision, Cheat Sheet, and Self-Check Questions

Chapter 1 — The Platform: Rounds, Divisions, and Problem Difficulty

1.1 Contest types

Type	What it is	Who it's for
Div. 3 / Div. 4	Easier rated rounds, slower-paced problem difficulty curve	Newer competitors, rating roughly up to 1600-1900
Div. 2	Standard rated round	Rating roughly up to 2100 (often with an extended range)
Div. 1	Harder rated round	Rating roughly 1900+
Div. 1 + Div. 2 combined	Same problem set, different starting letters shown to each division	Everyone, split by rating
Educational rounds	Rated for a rating range, problems chosen to teach a technique	Practice-oriented, has strong editorials
Global Rounds	Open to all ratings, larger and more varied problem sets	Everyone

1.2 How problems are labeled

Within one contest, problems are lettered A, B, C, D, E, F... in roughly increasing difficulty order (with occasional exceptions). Each problem also carries a numeric difficulty rating (e.g. 1200, 1700) assigned after the contest based on how solvers actually performed — this is usually a more reliable difficulty signal than the letter alone, especially across different divisions.

✓ Filter by problem rating, not just by letter, when practicing

“Solve some C's” is a weaker practice signal than “solve problems rated 1300” — the numeric rating accounts for difficulty drift between contests and divisions, which a fixed letter doesn't.

1.3 Where to find problems by difficulty

Tool	Use for
Codeforces Problemset, filtered by rating tag	Targeted practice at a specific difficulty
Codeforces "Problems" page tag filters (dp, graphs, greedy, etc.)	Practicing a specific technique across many difficulties
A rating-tracking tool (many community-built ones exist)	Seeing your solved/unsolved breakdown by rating band

** Quick Revision**

- Divisions determine problem difficulty range and starting letter, not skill in an absolute sense — the numeric problem rating is the more reliable difficulty signal.
- Educational rounds and their editorials are particularly good for deliberate, technique-targeted practice.
- Filter practice by problem RATING, not just by letter, once you've done a handful of contests.

Chapter 2 — How the Rating System Works

2.1 The concept, without the formula

Codeforces uses an Elo-derived system: before a contest, every participant's rating predicts roughly how they'll place relative to everyone else. After the contest, your ACTUAL placement is compared to that prediction — placing better than predicted raises your rating, placing worse lowers it. The exact formula isn't necessary to memorize; what matters practically is what drives the direction and size of the change.

Factor	Effect on rating change
Placing better than your rating predicted	rating increases
Placing worse than your rating predicted	rating decreases
Facing a field with many strong/high-rated competitors and doing well	larger increase than the same placement against a weaker field
A single very bad (or very good) contest	moves rating less than intuition suggests — the system is deliberately somewhat conservative per-contest

2.2 Rating bands (informal, community convention)

Rating range	Common informal name
< 1200	Newbie
1200 - 1399	Pupil
1400 - 1599	Specialist
1600 - 1899	Expert
1900 - 2099	Candidate Master
2100 - 2299	Master

Rating range	Common informal name
2300+	International Master / Grandmaster tiers

⚠ These names and exact cutoffs are a platform convention, not a law of skill

Exact boundaries have shifted historically and could shift again. Treat this table as orientation, not as something to memorize precisely — what matters far more is the skill content of Chapter 5's band-by-band map.

2.3 Why rating is a noisy but honest signal

What rating IS	What rating is NOT
A rough estimate of contest problem-solving performance under time pressure	A precise measure of DSA knowledge alone
Sensitive to speed and accuracy, not just correctness eventually	The same thing as being able to solve a problem untimed, with unlimited attempts
Something that moves gradually with consistent practice	Something one great or one bad contest should swing wildly

Quick Revision

- Rating change depends on how your actual placement compares to your PREDICTED placement, not on an absolute score.
- A stronger field gives bigger rating swings for the same relative placement.
- Rating measures contest performance (speed + accuracy under time pressure), which is related to but not identical to raw DSA knowledge.

Chapter 3 — How to Practice: Virtual Contests, Upsolving, Editorials

3.1 Three practice modes, and when to use each

Mode	What it is	Best for
Live rated contest	Participate in a real contest as it happens	Testing yourself under genuine time pressure and rating stakes
Virtual contest	Replay a past contest with the same time limit, as if live (unrated)	Practicing contest pacing and pressure without rating risk
Untimed practice / upsolving	Solve problems from past contests with no clock	Deliberate skill-building on specific techniques or difficulty bands

✓ Virtual contests are underused

A virtual contest reproduces the actual time pressure and problem ordering of a real round without touching your rating — it's the closest thing to “practice under game conditions” available, and is especially valuable while still building contest-pacing instincts (Chapter 4).

3.2 Upsolving and editorials

Upsolving means going back after a contest and solving the problems you didn't finish in time, using no time limit. This is where most actual skill growth happens — contests test what you already know; upsolving is where you learn what you didn't.

Step	Why it matters
Attempt the problem yourself first, even after the contest ends	Struggling productively before reading the editorial builds the pattern-recognition instinct Part 21 described
Read the editorial only after a genuine attempt (or being fully stuck for a while)	Reading too early replaces problem-solving practice with reading comprehension practice

Step	Why it matters
After reading, re-implement from scratch without looking	Confirms you actually understood the idea, not just recognized it while reading
Note which CONCEPT the problem was really testing	Builds the clue-word → pattern mapping from Part 21, Chapter 5, from real examples

⚠ Reading the editorial immediately after getting stuck is a common trap

It feels productive but mostly builds “I can follow an explanation” skill, not “I can generate the idea myself” skill — the second is what contests actually test. Give yourself real stuck-time first.

Practice — Chapter 3

Problem 3.1 — Design your own upsolving checklist [Level 1]

Using the table in 3.2, write a personal checklist you'll follow every time you upsolve a problem after a contest.

Hint 1: Include a rule for how long you'll stay stuck before reading the editorial, and a rule for re-implementing without looking afterward.

Approach: This is a self-reflection exercise — there's no single correct checklist, but it should include: genuine attempt, a defined stuck-time threshold, re-implementation from scratch, and noting the underlying concept.

Complexity: N/A — a process exercise, not an algorithmic one.

Chapter 4 — Contest Strategy: Reading, Ordering, Time Allocation

4.1 Don't always solve in letter order

Problems get harder roughly in letter order, but “roughly” matters — a slow warm-up on A can cost more than the problem is worth, and sometimes a later problem in your comfort zone (a known pattern from Part 21's catalog) is faster to solve than the one right in front of you.

Situation	Suggested response
Stuck on the current problem for several minutes with no progress	Skim the next problem — sometimes a fresh problem unsticks your thinking, or is simply easier for you personally
Problem statement is long and you haven't fully read it	Read the WHOLE statement, including constraints and sample explanations, before starting to code — skimming causes expensive misunderstandings
You immediately recognize the pattern (Part 21) in a problem	Prioritize it, even if it's technically a 'later' letter, if you're confident

4.2 Reading a statement well, every time

Read this section	Specifically checking for
Problem statement body	What's actually being asked — restate it in your own words
Input format	Exact order and format values arrive in — misreading this causes silent wrong answers
Constraints	Target complexity (Part 21, Chapter 2) — read this BEFORE designing an approach
Sample input/output	Verify your understanding by tracing the sample by hand
Sample explanation (if given)	Often reveals an edge case or a subtlety the main statement didn't make obvious

⚠ Time lost to a misread statement is the most avoidable kind of time loss

Coding a fast, correct solution to the WRONG problem (because of a misread constraint or input format) costs more time than reading carefully the first time — this is worth treating as a fixed, non-negotiable step, not something to rush under pressure.

4.3 Implementation speed vs. debugging speed

Skill	How it's built	What slows it down
Implementation speed	A memorized, muscle-memory template (fast I/O, common includes) and lots of reps	Re-deriving syntax you should already know cold, from Parts I-IV
Debugging speed	Systematic habits: check edge cases first, print intermediate state, re-read constraints for overflow/off-by-one risk	Randomly changing code and re-submitting without a hypothesis

📝 Quick Revision

- Reading the full statement, input format, constraints, and samples before coding is non-negotiable — misreads are the costliest, most avoidable time loss.
- Problem order (by letter) is a rough guide, not a rule — skip ahead if you're stuck and recognize an easier pattern elsewhere.
- Implementation speed comes from a memorized template and repetition; debugging speed comes from systematic habits, not guessing.

Chapter 5 — The Rating-by-Rating Skill Map (800 → 2000+)

For each band: the concepts typically required, the patterns that show up constantly, and what usually blocks the jump to the next band. This maps roughly onto Parts already covered in this book.

5.1 Rating 800 - 1000

Aspect	Detail
Concepts required	Basic I/O, conditions, loops, arrays, simple math (Parts I-III)
Typical patterns	Direct simulation, simple counting, basic string manipulation
What blocks the next jump	Implementation slips (off-by-one, wrong data type) more often than not knowing the idea
How to move up	Solve MANY problems at this level fast and cleanly; speed and fewer bugs matter more here than new theory

5.2 Rating 1000 - 1300

Aspect	Detail
Concepts required	Sorting, basic greedy, two pointers, prefix sums, STL fluency (Parts III-VI)
Typical patterns	Sort-then-greedy, simple two-pointer, frequency counting with maps
What blocks the next jump	Not yet reading constraints to pick the right complexity class before coding (Part 21, Chapter 2)
How to move up	Deliberately practice the 'constraints → complexity' habit; start timing yourself

5.3 Rating 1300 - 1600

Aspect	Detail
Concepts required	Binary search (including on the answer), basic DP, BFS/DFS, basic graph representations (Parts VI, X-XI, XIII, XV)
Typical patterns	Binary search on a monotonic condition, simple 1D/2D DP, shortest path on an unweighted graph
What blocks the next jump	DP state design — identifying WHAT the state needs to represent is often the real difficulty, not the transition
How to move up	Deliberate DP practice: for every DP problem solved, explicitly write out state/transition/base case (Part XV's rhythm) even after solving it

5.4 Rating 1600 - 1900

Aspect	Detail
Concepts required	DSU, Dijkstra, more DP variants (knapsack family, digit-ish reasoning), number theory basics (Parts XIII, XV-XVI)
Typical patterns	Graph problems needing a specific algorithm choice (not just BFS/DFS), DP with a non-obvious state dimension
What blocks the next jump	Problems increasingly need a NON-OBVIOUS observation before a known technique applies — straightforward pattern matching alone stops being enough
How to move up	Practice the full brute-force → bottleneck → observation → optimal chain (Part 21) deliberately, even on problems you eventually solve

5.5 Rating 1900 - 2000+

Aspect	Detail
Concepts required	Segment/Fenwick trees, more advanced graph algorithms, stronger number theory and combinatorics, bitmask DP (Parts XVIII-XIX, Part 19)

Aspect	Detail
Typical patterns	Combining two or more techniques in a single problem (e.g. binary search + a data structure; DP + bitmask)
What blocks the next jump	Speed of correctly IMPLEMENTING a correct idea becomes a real bottleneck, not just having the idea
How to move up	Build a fast, memorized template library (segment tree, DSU, Dijkstra) so implementation stops being the slow part of a contest

✍ **This progression is not strictly linear in practice**

Individual contests are noisy, and everyone's actual strengths cut across these bands unevenly — someone might have strong DP but weak graph fundamentals, for instance. Use this table to identify gaps in your OWN profile rather than assuming you must have mastered every row of a band before the next one is reachable.

✍ **Quick Revision**

- Each rating band adds specific concepts, but also raises expectations on speed and implementation cleanliness at the PREVIOUS band's concepts.
- Below ~1300, implementation cleanliness and speed matter more than new theory; above ~1600, non-obvious observations start to matter more than technique recall alone.
- Progress is not linear or uniform across topics — identify your own specific gaps rather than assuming strict mastery-by-band.

Chapter 6 — Upsolving Training: Turning a Failure Into a Skill

6.1 The five questions to ask about every failed problem

Question	Why it matters
What did I try first, and why did it fail?	Names your default instinct, so you notice if it's systematically wrong
What was the observation I missed?	The single most valuable thing to extract — write it down in your own words
How could I have discovered that observation myself?	Turns a one-off fact into a repeatable strategy for future problems
What concept/pattern did this problem actually test?	Feeds directly into Part 21's clue-word catalog — build your own examples list over time
Would I recognize this pattern faster next time?	The real measure of whether upsolving worked

6.2 Worked example of this process

Question	Example answer
What did I try first?	Brute-forced pairwise comparisons — timed out on large n
What was missed?	The condition was monotonic in a hidden variable, which meant binary search on the answer applied
How to discover it myself next time?	Whenever a problem asks for a 'minimum X such that a property holds', check whether the property is monotonic BEFORE assuming brute force is the only way
What concept was tested?	Binary search on the answer (Part VI)
Faster next time?	Yes — 'minimum X such that' is now a flagged clue phrase

6.3 Keeping a personal pattern log

A simple running list — problem, clue phrase, technique, what you missed — turns individual contest losses into a compounding personal reference, and after enough entries starts to look a lot like your own version of Part 21's catalog, built from problems you personally struggled with rather than generic examples.

✓ This log is worth more than it looks like early on

The value compounds: entry #50 in your own log, in your own words, tied to a problem you personally struggled with, tends to stick far better than the same fact read generically in a book — including this one.

📌 Quick Revision

- The five-question process turns a failed problem into a reusable insight, not just a solved problem.
- Write down HOW you could have discovered the missed observation yourself — that's what actually transfers to future problems.
- A personal pattern log, built from your own failures over time, tends to be more durable than memorizing someone else's catalog.

Practice — Chapter 6

Problem 6.1 — Run the five-question process on a real problem [Level 2]

Pick a problem you solved incorrectly or too slowly in a past contest (or virtual contest), and answer all five questions from Section 6.1 in writing.

Hint 1: Be specific about the observation you missed — vague answers like 'I should have thought harder' don't transfer to future problems.

Approach: This is a reflective exercise best done immediately after a real contest, while the problem is still fresh.

Complexity: N/A.

Part 22 — Master Revision

Rating-by-Rating Skill Map, Compressed

Band	Core focus	What usually blocks the next jump
800-1000	Clean, correct basics	Implementation slips, not missing knowledge
1000-1300	Sorting, greedy, two pointers, STL	Not reading constraints before choosing complexity
1300-1600	Binary search, basic DP, BFS/DFS	DP state design specifically
1600-1900	DSU, Dijkstra, deeper DP, number theory	Problems need a non-obvious observation, not just pattern recall
1900-2000+	Segment/Fenwick trees, combined techniques, bitmask DP	Implementation speed of correct ideas becomes the bottleneck

Must Remember

Quick Revision

- Rating change depends on placement relative to PREDICTION, not an absolute score — a stronger field means bigger swings for the same relative placement.
- Virtual contests give real time pressure without rating risk — an underused practice mode.
- Upsolving is where most actual skill growth happens; give yourself genuine stuck-time before reading an editorial, and re-implement afterward without looking.
- Read the full statement, input format, constraints, and samples before coding — misreads are the costliest avoidable time loss in a contest.
- Each rating band raises expectations on speed/cleanliness at PREVIOUS concepts, not just adding new ones.
- The five-question upsolving process (what failed, what was missed, how to find it myself, what concept, would I be faster now) turns losses into durable, transferable skill.

Common Mistakes Recap

Mistake	Fix
Reading an editorial immediately after getting stuck	Give yourself real stuck-time first — reading too early trains reading comprehension, not problem-solving
Solving strictly in letter order even when stuck	Skim ahead — a later problem may match a pattern you recognize faster
Skimming the constraints or input format under time pressure	Treat a full read as a fixed, non-negotiable step — misreads cost more time than they save
Judging progress by a single contest's rating swing	The system is deliberately conservative per-contest; judge trends over many contests instead
Upsolving without writing down what was actually missed	Use the five-question process (Chapter 6) — vague reflection doesn't transfer to future problems

Mistake	Fix
Assuming strict band-by-band mastery is required to progress	Identify YOUR specific gaps — real skill profiles are uneven across topics

Self-Check Questions

- Why can placing at the same absolute rank in two different contests produce very different rating changes?
- Why does upsolving typically produce more skill growth than the live contest itself?
- Why is reading the full problem statement and constraints before coding considered non-negotiable, even under time pressure?
- Why do rating bands above ~1600 rely increasingly on 'non-obvious observations' rather than direct technique recall?
- Why might writing down HOW you could have discovered a missed observation matter more than writing down the observation itself?

Cheat Sheet — Part 22

```
// Contest checklist (run through this at the start of every
problem)
// 1. Read the FULL statement, including input format and
samples
// 2. Note the constraints -> target complexity (Part 21, Ch. 2)
// 3. Trace a sample by hand to confirm understanding
// 4. Find the brute force, identify the bottleneck
// 5. Match the bottleneck to a known pattern (Part 21, Ch. 5)
// 6. Implement using a memorized, fast template
// 7. Test edge cases BEFORE submitting, not just the given
samples

// Upsolving checklist (run this after every contest)
// 1. Attempt unsolved problems again, untimed, before reading
editorials
// 2. Give yourself real stuck-time -- don't read the editorial
too early
// 3. After reading, re-implement from scratch without looking
// 4. Log: problem, clue phrase, technique, what was missed
```

Coming up next

LeetCode preparation (Easy → Medium → Hard, pattern-based practice sets) is next in the roadmap, followed by interview preparation and mock contests. Say which you'd like built next and under what part number.